

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Modul Belajar 1

Kenalan dengan KMP dan Siapkan Alat

Kotlin seperlunya, struktur proyek Kotlin Multiplatform, dan aplikasi pertama yang berjalan di lebih dari satu platform

Muhammad Habib Algifari, S.Kom., M.T.I.

Pendamping Pertemuan 1. Tahun 2026

Komponen	Keterangan
Pertemuan	1
Capaian	CPMK0502, menjelaskan konsep pemrograman untuk pengembangan perangkat lunak
Waktu belajar	sekitar 4 jam belajar mandiri, ditambah 2,5 jam di kelas

MODUL 1

Peta belajar

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

Urutan	Bagian
1	Tentang modul ini
2	Masalah pembuka: proyektor mati, lalu lapor ke mana?
3	Langkah 1: Nilai yang tetap dan nilai yang berubah
4	Langkah 2: Fungsi dan string template
5	Langkah 3: Memilih dan mengulang
6	Langkah 4: Apa itu Kotlin Multiplatform
7	Langkah 5: Kode bersama dan kode khusus
8	Langkah 6: expect berjanji, actual menepati
9	Langkah 7: Siapkan alat
10	Langkah 8: Halo dari Laporkampus
11	Latihan mandiri
12	Bawa ke proyekmu
13	Rangkuman
14	Cek pemahaman
15	Bacaan lanjut dan persiapan minggu depan

PEMBUKA

Tentang modul ini

Modul ini menemani Pertemuan 1. Isinya sama dengan slide di kelas, tetapi ditulis lebih pelan dan lebih lengkap, supaya kalian bisa mengulang sendiri di rumah tanpa harus menebak maksud slide. Bacalah berurutan dari Langkah 1 sampai Langkah 8. Setiap langkah dibangun di atas langkah sebelumnya.

Setelah menyelesaikan modul ini, kalian bisa

No	Kemampuan	Dibahas di
1	Membaca dan menulis Kotlin dasar: val, var, tipe data, fungsi, string template, when, if, dan for	Langkah 1 sampai 3
2	Menjelaskan ide Kotlin Multiplatform: apa yang ditulis sekali di commonMain dan apa yang tetap ditulis per platform	Langkah 4 sampai 6
3	Menjalankan proyek KMP pertama di emulator Android atau di desktop	Langkah 7 dan 8

Ketiga kemampuan ini mendukung **CPMK0502**: menjelaskan konsep pemrograman untuk pengembangan perangkat lunak. Karena itu, yang dinilai minggu ini bukan hanya aplikasinya jalan, tetapi juga kalian bisa menjelaskan isi foldernya.

Cara memakai modul ini

Setiap langkah punya pola yang sama. Ada penjelasan konsep, lalu contoh dari kehidupan sehari-hari, lalu kode yang bisa langsung kalian ketik. Di beberapa tempat ada kotak **Tebak dulu**. Tulis tebakan kalian di kertas sebelum membaca jawabannya. Kebiasaan menebak lalu membuktikan ini yang membuat konsep menempel, bukan sekadar membaca.

PENTING Ketik, jangan salin

Ketik ulang setiap contoh kode di Android Studio atau di Kotlin Playground. Saat mengetik, kalian akan membuat salah ketik kecil, membaca pesan error, lalu memperbaikinya. Proses itulah yang melatih kalian membaca kode.

Yang perlu disiapkan

Laptop dengan RAM minimal 8 GB dan ruang kosong 20 GB, koneksi internet yang stabil untuk unduhan pertama, dan akun GitHub. Untuk Langkah 1 sampai 3, kalian belum perlu memasang apa pun: cukup buka **play.kotlinlang.org** di browser.

PEMBUKA

Masalah pembuka: proyektor mati, lalu lapor ke mana?

Bayangkan proyektor di ruang kuliah mati sepuluh menit sebelum kelas dimulai. Kalian kirim pesan ke grup angkatan. Pesannya tertimbun obrolan lain, dan tidak ada yang tahu apakah sudah ada petugas yang menangani.

Laporan seperti ini butuh tempat yang jelas. Setiap laporan punya nomor, punya lokasi, kalau perlu punya foto, dan statusnya bisa dipantau oleh pelapor: masih baru, sedang diproses, atau sudah selesai.

Sepanjang semester kita akan membangun aplikasi untuk masalah itu. Namanya **LaporKampus**. Setiap minggu satu konsep baru langsung dipasang di aplikasi yang sama, jadi kalian melihat kenapa sebuah konsep dibutuhkan, bukan hanya cara menulisnya.

Pertemuan	Yang bertambah di LaporKampus
1	Kerangka aplikasi yang jalan di Android dan desktop
2	Model data laporan dan perubahan status
3	Layar daftar laporan
4	Form laporan baru dengan ViewModel
5	Pindah layar dan menu bawah
6	Daftar laporan diambil dari server
7	Draf laporan tetap aman saat offline
8	Foto kerusakan, lokasi, dan izin akses
9	AI menebak kategori dan prioritas laporan
10	Test, dependency injection, dan debugging

TEBAK DULU Coba pikirkan

Kalau kalian yang membuat aplikasinya, data apa saja yang wajib dicatat dari satu laporan kerusakan? Tulis minimal lima.

JAWABAN Jawaban yang biasa muncul

Lokasi, jenis kerusakan, foto, waktu lapor, nama pelapor, dan status. Simpan daftar kalian. Di Pertemuan 2, daftar ini akan kita ubah menjadi kode, yaitu data class bernama Laporan.

LANGKAH 1

Nilai yang tetap dan nilai yang berubah

Program menyimpan data di dalam **variabel**. Bayangkan variabel sebagai kotak berlabel. Labelnya adalah nama variabel, isinya adalah nilainya.

Kotlin punya dua jenis kotak:

Kata kunci	Artinya	Contoh sehari-hari
<code>val</code>	Sekali diisi, isinya tidak bisa diganti	Tanggal lahir kalian
<code>var</code>	Isinya boleh diganti kapan saja	Saldo dompet digital kalian

Main.kt

```
val kampus = "ITERA"           // tidak bisa diganti
var jumlahLaporan = 0          // boleh berubah
jumlahLaporan += 1              // sekarang isinya 1
```

Tipe data

Setiap nilai punya tipe. Tipe menentukan nilai seperti apa yang boleh masuk ke sebuah kotak. Empat tipe yang paling sering kita pakai:

Tipe	Isinya	Contoh
<code>String</code>	Teks	"GKU 1 R201"
<code>Int</code>	Bilangan bulat	3
<code>Double</code>	Bilangan pecahan	4.5
<code>Boolean</code>	Benar atau salah	true, false

Tipe boleh ditulis jelas setelah titik dua, boleh juga tidak ditulis. Kalau tidak ditulis, compiler menebaknya dari nilai awal.

Main.kt

```
val ruang: String = "GKU 1 R201" // tipe ditulis jelas
val prioritas = 3                 // compiler menebak: Int
val sudahDibaca: Boolean = false
val skor = 4.5                   // compiler menebak: Double
```

TEBAK DULU

Apa yang terjadi kalau di baris berikutnya kita menulis `kampus = "UNILA"`?

JAWABAN

Kode gagal dikompilasi dengan pesan **Val cannot be reassigned**. `kampus` dibuat dengan `val`, jadi isinya terkunci. Compiler menolak sebelum program sempat berjalan, dan ini hal yang baik: kesalahan tertangkap lebih awal.

Nilai yang boleh kosong

Kadang sebuah data memang belum ada. Misalnya, laporan boleh dikirim tanpa foto. Di Kotlin, variabel biasa tidak boleh berisi `null` (kosong). Kalau kita ingin mengizinkan kosong, tambahkan tanda tanya di belakang tipenya.

Main.kt

```
val namaFoto: String? = null           // boleh kosong
val panjang = namaFoto?.length        // hasilnya null, bukan error
val tampil = namaFoto ?: "Tanpa foto"
println(tampil)                       // Tanpa foto
```

`?.` artinya ambil nilainya hanya kalau tidak kosong. `?:` artinya kalau kosong, pakai nilai cadangan di sebelah kanan. Topik ini kita perdalam di Pertemuan 2. Untuk sekarang cukup kenali bentuknya, karena latihan 1 memakainya.

PENTING Kebiasaan baik

Pakai `val` sebanyak mungkin, dan pakai `var` hanya kalau nilainya memang harus berubah.

Semakin sedikit nilai yang bisa berubah, semakin mudah kita menebak isi program. Kebiasaan ini akan sangat terasa di Pertemuan 4 saat membahas state.

LANGKAH 2

Fungsi dan string template

Fungsi adalah sekumpulan perintah yang diberi nama, supaya bisa dipanggil berulang kali.

Analoginya seperti resep: sekali ditulis, bisa dipakai memasak berkali-kali dengan bahan yang berbeda.

Main.kt

```
fun sapa(nama: String): String {
    return "Halo $nama, ada $jumlahLaporan laporan baru"
}
```

Mari baca baris pertamanya pelan-pelan:

Bagian	Artinya
fun	Kita sedang membuat fungsi
sapa	Nama fungsinya
(nama: String)	Fungsi ini menerima satu masukan bernama nama, bertipe String
: String	Fungsi ini mengembalikan String
return	Nilai yang dikembalikan ke pemanggil

Fungsi satu baris

Kalau isi fungsi hanya satu ekspresi, kita bisa menulisnya dengan tanda sama dengan. Tipe kembaliannya pun boleh tidak ditulis karena ditebak compiler.

Main.kt

```
fun labelPrioritas(p: Int) = "Prioritas $p dari 5"
```

String template

Perhatikan tanda dolar di dalam teks. Itu namanya **string template**: cara menyisipkan nilai ke dalam teks tanpa menyambung dengan tanda tambah.

Penulisan	Kapan dipakai	Contoh
\$nama	Menyisipkan isi sebuah variabel	"Halo \$nama"
\${...}	Menyisipkan hasil perhitungan atau pemanggilan	"Total: \${jumlah * 2}"

Parameter bawaan

Parameter boleh punya nilai bawaan. Kalau pemanggil tidak mengisinya, nilai bawaan itu yang dipakai. Ini praktis untuk masukan yang biasanya sama. Program di bawah menggabungkan semua yang sudah kita bahas di langkah ini, jadi bisa langsung dijalankan utuh.

Main.kt

```

var jumlahLaporan = 1

fun sapa(nama: String): String {
    return "Halo $nama, ada $jumlahLaporan laporan baru"
}

fun labelPrioritas(p: Int) = "Prioritas $p dari 5"

fun buatLabel(lokalasi: String, gedung: String = "GKU 1") =
    "$gedung, $lokalasi"

fun main() {
    println(sapa("Rina"))
    println(labelPrioritas(3))
    println(buatLabel("R201"))
    println(buatLabel("Lab 3", gedung = "Labtek"))
}

```

Keluaran

```

Halo Rina, ada 1 laporan baru
Prioritas 3 dari 5
GKU 1, R201
Labtek, Lab 3

```

CONTOH Contoh sehari-hari

Fungsi buatLabel mirip formulir peminjaman ruang yang kolom gedungnya sudah terisi GKU 1. Kalau ruangnya memang di GKU 1, kalian cukup mengisi nomor ruang. Kalau di gedung lain, baru kolom gedungnya ditimpa.

LANGKAH 3

Memilih dan mengulang

Program jarang berjalan lurus dari atas ke bawah. Ia perlu **memilih** jalan berdasarkan kondisi dan **mengulang** pekerjaan yang sama.

if untuk dua pilihan

Main.kt

```

val prioritas = 5
if (prioritas == 5) {
    println("Kabari petugas sekarang")
} else {
    println("Masukkan antrean")
}

```

when untuk banyak pilihan

Kalau pilihannya lebih dari dua, `when` jauh lebih rapi daripada `if` yang bertumpuk. `when` mengecek cabang dari atas, menjalankan cabang pertama yang cocok, lalu berhenti. Kerennya, `when` juga bisa menghasilkan nilai.

Main.kt

```
fun tingkat(p: Int): String = when (p) {
    5 → "Darurat"
    in 3..4 → "Tinggi"
    else → "Biasa"
}
```

`3..4` adalah **rentang** dari 3 sampai 4, termasuk kedua ujungnya. `in 3..4` artinya nilainya ada di dalam rentang itu. Cabang `else` menangkap semua nilai yang tidak cocok dengan cabang di atasnya.

CONTOH Contoh sehari-hari

`when` bekerja seperti petugas triase di klinik kampus. Pasien diperiksa dari kriteria paling gawat dulu. Begitu cocok dengan satu kriteria, pasien langsung diarahkan dan pemeriksaan berhenti. Yang tidak masuk kriteria mana pun masuk antrean biasa, sama seperti cabang `else`.

for untuk mengulang

Main.kt

```
fun main() {
    for (p in 1..5) {
        if (p == 5) println("Kabari petugas sekarang")
        println("$p: ${tingkat(p)}")
    }
}
```

TEBAK DULU

Tulis tiga baris terakhir yang dicetak program di atas.

JAWABAN

Keluaran

```
4: Tinggi
Kabari petugas sekarang
5: Darurat
```

Saat p sama dengan 5, perintah if dijalankan lebih dulu karena letaknya di atas println. Itu sebabnya Kabari petugas sekarang muncul sebelum 5: Darurat. Urutan baris di dalam badan perulangan menentukan urutan keluaran.

PENTING **Buktikan sendiri**

Buka play.kotlinalang.org, tempel fungsi tingkat dan main di atas, lalu tekan Run. Setelah itu ubah rentangnya menjadi 1..7. Baris apa yang bertambah? Jawabannya 6: Biasa dan 7: Biasa, karena keduanya jatuh ke cabang else.

LANGKAH 4

Apa itu Kotlin Multiplatform

Biasanya, kalau sebuah aplikasi harus ada di Android dan iPhone, perusahaan membentuk dua tim. Tim Android menulis dengan Kotlin, tim iOS menulis dengan Swift. Aturan bisnis yang sama ditulis dua kali, dites dua kali, dan sering kali hasilnya sedikit berbeda.

Kotlin Multiplatform (KMP) dari JetBrains menawarkan cara lain. Logika aplikasi ditulis sekali dengan Kotlin, lalu dijalankan di Android, iOS, dan desktop. Lewat **Compose Multiplatform**, tampilannya pun bisa ikut dibagi. Kode yang memang berbeda per platform, seperti kamera atau sensor, tetap bisa ditulis terpisah. Hasil akhirnya tetap aplikasi native, bukan halaman web yang dibungkus.

ANALOGI **Satu resep, banyak wadah**

Resep pindang patin tetap sama, mau dimakan di warung atau dibungkus ke kos. Yang berbeda hanya wadahnya. Di LaporKampus, cara mencatat dan memproses laporan adalah resepnya. Android, iOS, dan desktop adalah wadahnya.

Kenapa ini penting bagi kita

Ada tiga keuntungan yang akan kalian rasakan sepanjang semester. Pertama, aturan seperti status laporan hanya boleh maju dari Baru ke Diproses ke Selesai cukup ditulis sekali, jadi tidak mungkin berbeda antara versi Android dan iOS. Kedua, test untuk aturan itu juga cukup ditulis sekali. Ketiga,

kalian yang tidak punya Mac tetap bisa membuat dan mencoba sebagian besar aplikasi lewat target Android dan desktop.

LANGKAH 5

Kode bersama dan kode khusus

Proyek KMP dibagi menjadi beberapa folder sumber yang disebut **source set**. Setiap source set punya peran yang jelas.

Source set	Isinya	Contoh di Laporkampus
<code>commonMain</code>	Kode untuk semua platform	Model laporan, aturan status, pemanggilan server, sebagian besar UI
<code>androidMain</code>	Kode khusus Android, boleh memakai Android SDK	Kamera dan izin akses versi Android
<code>iosMain</code>	Kode khusus iOS, boleh memakai iOS SDK	Kamera dan izin akses versi iOS
<code>desktopMain</code>	Kode khusus desktop di JVM	Jendela aplikasi dan akses berkas

Aturan memilih foldernya sederhana. Tanyakan: apakah kode ini sama untuk semua platform? Kalau sama, taruh di `commonMain`. Kalau berbeda karena memakai fitur bawaan sistem operasi, taruh di folder platformnya.

TEBAK DULU

Aturan bahwa status laporan hanya boleh maju dari Baru ke Diproses ke Selesai, ditaruh di folder mana?

JAWABAN

Di **`commonMain`**. Aturannya sama persis di semua platform dan tidak butuh fitur khusus sistem operasi. Sebaliknya, kode untuk membuka kamera ditaruh di `androidMain` dan `iosMain`, karena cara membuka kamera di Android dan iOS memang berbeda.

Struktur folder proyek

Seperti inilah isi proyek Laporkampus setelah dibuat dari template:

Struktur folder

```
LaporKampus/
  composeApp/
    src/
      commonMain/kotlin/
      androidMain/kotlin/
      iosMain/kotlin/
      desktopMain/kotlin/
  iosApp/
  gradle/libs.versions.toml
  build.gradle.kts
  settings.gradle.kts
```

Berkas atau folder	Gunanya
composeApp	Modul utama aplikasi, berisi semua source set
iosApp	Proyek Xcode untuk build iOS, hanya dibuka di Mac
libs.versions.toml	Daftar versi library di satu tempat
build.gradle.kts	Pengaturan build, termasuk target platform

PENTING Catatan versi

Pada template KMP yang lebih baru, folder desktopMain bisa bernama **jvmMain**. Isinya sama, hanya namanya yang berbeda. Proyek hands-on di repo mata kuliah tidak menyertakan folder iosApp. Untuk mencoba iOS, buat proyek baru lewat kmp.jetbrains.com.

LANGKAH 6

expect berjanji, actual menepati

Bagaimana kode di commonMain memakai sesuatu yang berbeda per platform? Caranya dengan pasangan kata kunci **expect** dan **actual**.

Di commonMain kita menulis **expect**: sebuah janji bahwa fungsi ini ada, tanpa isinya. Setiap platform lalu wajib menepati janji itu dengan **actual** miliknya sendiri.

```
commonMain/Platform.kt

expect fun namaPlatform(): String

fun sapaan(): String =
    "LaporKampus berjalan di ${namaPlatform()}"
```

```
androidMain/Platform.android.kt
```

```
import android.os.Build

actual fun namaPlatform(): String =
    "Android ${Build.VERSION.SDK_INT}"
```

iosMain/Platform.ios.kt

```
import platform.UIKit.UIDevice

actual fun namaPlatform(): String =
    UIDevice.currentDevice.systemName()
```

desktopMain/Platform.desktop.kt

```
actual fun namaPlatform(): String = "Desktop JVM"
```

Fungsi `sapaan()` ditulis sekali di `commonMain`. Hanya `namaPlatform()` yang isinya berbeda di tiap platform.

ANALOGI Janji dan pelunasan

expect itu seperti janji di kontrak: akan ada fungsi `namaPlatform`. actual adalah pelunasan janji di setiap platform. Compiler berperan sebagai penagih. Kalau ada satu platform saja yang belum melunasi, build untuk platform itu ditolak.

TEBAK DULU

Kode di atas dijalankan dengan konfigurasi desktop. Kalimat apa yang tampil? Lalu, apa yang terjadi kalau kita lupa menulis actual di `iosMain`?

JAWABAN

Yang tampil adalah **LaporKampus berjalan di Desktop JVM**. Kalau actual di `iosMain` hilang, build target iOS gagal karena deklarasi expect belum punya pasangan actual. Build Android dan desktop tetap berhasil, karena keduanya sudah menepati janjinya.

LANGKAH 7

Siapkan alat

Sekarang kita pindah dari browser ke laptop. Periksa dulu kebutuhan minimalnya.

Kebutuhan	Minimal
RAM	8 GB, disarankan 16 GB
Ruang kosong	20 GB
Sistem operasi	Windows 10 atau 11, macOS 12 ke atas, atau Linux
Perangkat lunak	JDK 17 ke atas, Android Studio, plugin Kotlin Multiplatform, dan Git
Build iOS	Mac dengan Xcode, opsional

PENTING Kalau RAM kalian 8 GB

Emulator Android akan terasa berat. Jalankan target desktop lebih dulu karena jauh lebih ringan. Emulator bisa dicoba belakangan, atau pakai HP Android sendiri lewat kabel USB.

Lima langkah sampai proyek siap

1. **Unduh Android Studio** dari developer.android.com/studio, lalu pasang seperti aplikasi biasa.
2. **Pasang komponen SDK.** Saat wizard pertama kali muncul, pilih Android SDK, SDK Platform, dan Android Virtual Device.
3. **Pasang plugin KMP.** Buka Settings, lalu Plugins, cari Kotlin Multiplatform, lalu Install dan restart Android Studio.
4. **Buat proyek baru** di kmp.jetbrains.com. Isi nama `LaporKampus`, paket `id.ac.itera.laporkampus`, centang target Android, iOS, dan desktop, lalu unduh berkas zip-nya dan ekstrak.
5. **Buka proyek dan tunggu Gradle sync.** Di Android Studio pilih Open, arahkan ke folder hasil ekstrak. Unduhan pertama bisa memakan waktu lama, jadi pastikan koneksi stabil dan jangan menutup Android Studio di tengah jalan.

Kalau Gradle bermasalah

Gradle adalah alat yang mengunduh library dan membangun aplikasi. Di awal, ia sering rewel. Ini empat masalah yang paling sering muncul:

Pesan error	Penyebab	Yang dicoba
Gradle sync failed	JDK atau SDK tidak ditemukan	Periksa <code>JAVA_HOME</code> dan <code>ANDROID_HOME</code> , atau atur JDK di Settings, Build Tools, Gradle
SDK location not found	Berkas <code>local.properties</code> belum ada	Buat berkas <code>local.properties</code> berisi <code>sdk.dir=</code> lalu lokasi Android SDK
Kotlin version mismatch	Versi Kotlin dan Compose tidak cocok	Samakan versi di <code>libs.versions.toml</code>

Pesan error	Penyebab	Yang dicoba
iOS build failed	Xcode belum terpasang atau sudah usang	Pasang Xcode terbaru di Mac

PENTING Kalau error tidak ada di tabel

Salin pesan error lengkap, dari baris pertama sampai terakhir, lalu kirim ke grup kelas atau ke asisten. Jangan mengirim tangkapan layar yang terpotong, karena penyebabnya sering ada di baris yang tidak ikut terfoto.

LANGKAH 8

Halo dari LaporKampus

Buka berkas `App.kt` di `commonMain`, lalu ganti isinya dengan kode berikut.

`commonMain/App.kt`

```
import androidx.compose.animation.AnimatedVisibility
import androidx.compose.foundation.layout.Column
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment

@Composable
fun App() = MaterialTheme {
    var tampil by remember { mutableStateOf(false) }
    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Text("LaporKampus")
        Button(onClick = { tampil = !tampil }) {
            Text("Cek platform")
        }
        AnimatedVisibility(tampil) { Text(sapaan()) }
    }
}
```

Kalian tidak perlu memahami semua baris ini sekarang. Cukup kenali tiga hal. `@Composable` menandai fungsi yang menggambar tampilan. `Column` menyusun isinya dari atas ke bawah. Saat tombol ditekan, nilai `tampil` berubah, dan layar ikut berubah. Mekanisme di balik perubahan itu, yaitu state dan recomposition, kita bedah di Pertemuan 4.

TEBAK DULU

Sebelum menekan Run: apa yang tampil saat aplikasi pertama kali dibuka?

JAWABAN

Hanya judul Laporan Kampus dan tombol Cek platform. Kalimat sapaan belum tampil karena nilai awal `tampil` adalah false. Setelah tombol ditekan, sapaan muncul. Ditekan sekali lagi, sapaan hilang.

Menjalankan aplikasi

Platform	Caranya
Android	Pilih konfigurasi <code>composeApp</code> , pilih emulator atau HP, lalu tekan Run
Desktop	Paling cepat untuk mencoba tanpa emulator
iOS	Khusus Mac: pilih konfigurasi <code>iosApp</code> dan simulator, atau buka folder <code>iosApp</code> di Xcode

Kalian juga bisa menjalankannya lewat terminal dari folder proyek:

Terminal
<pre># Android, memasang ke emulator atau HP yang terhubung ./gradlew :composeApp:installDebug # Desktop ./gradlew :composeApp:run</pre>

Di Windows, ganti `./gradlew` dengan `gradlew.bat`.

PENTING Target minimal hari ini

Satu platform jalan. Kalau kalian berhasil menjalankan di dua platform dan bisa menjelaskan isi `commonMain` dan `androidMain`, kalian sudah memenuhi level tertinggi observasi praktik minggu ini.

PRAKTIK

Latihan mandiri

Ada tiga latihan di repo mata kuliah. Dua latihan pertama adalah pemanasan Kotlin. Latihan ketiga adalah latihan yang diobservasi.

Latihan	Lokasi	Yang dilatih
1	Kotlin Dasar, P1 Introduction to Kotlin Hands-on, handson1-latihan	Profil dengan string template, parameter bawaan, dan nilai yang mungkin null
2	Kotlin Dasar, P1 Introduction to Kotlin Hands-on, handson2-latihan	Mengubah nilai ujian menjadi huruf mutu dengan when dan rentang
3	P1 Pengenalan MK dan Setup Environment Hands-on, Handson1PlatformInfo.kt	Pesan sapaan yang menyebut nama platform lewat expect dan actual

CONTOH Petunjuk latihan 2

Mulailah dari nilai tertinggi, persis seperti fungsi tingkat di Langkah 3. Satu cabang untuk satu huruf mutu, misalnya `in 85..100 → "A"`, lalu cabang berikutnya untuk rentang di bawahnya sesuai tabel di soal latihan. Jangan lupa cabang else untuk nilai di luar 0 sampai 100. Uji fungsinya dengan nilai di batas rentang, misalnya 84 dan 85, karena di situlah kesalahan paling sering terjadi.

Yang diamati di latihan 3

Level	Skor	Yang terlihat
Sangat baik	85 sampai 100	Proyek jalan di dua target dan bisa menjelaskan isi <code>commonMain</code> dan <code>androidMain</code>
Baik	70 sampai 84	Proyek jalan di satu target, penjelasan struktur sebagian benar
Cukup	55 sampai 69	Proyek jalan setelah dibantu, belum bisa menjelaskan strukturnya
Kurang	di bawah 55	Environment belum terpasang sampai kelas selesai

Observasi ini dikerjakan individu dan bernilai 4% dari nilai akhir. Buktinya: push latihan ke repository latihan pribadi, lalu kirim tautan commit di LMS sebelum kelas selesai. Yang belum selesai di kelas boleh melanjutkan di rumah dan mengirim tautan sebelum pertemuan berikutnya, dengan level tertinggi Baik.

PRAKTIK

Bawa ke proyekmu

Kelompok dan tema sudah ditetapkan saat kontrak kuliah. Minggu ini setiap kelompok menyiapkan rumah kodenya.

1. **Buat repository.** Satu anggota membuat repository GitHub kelompok. Namanya memuat nomor tema, misalnya `pam-tema18-katalog-umkm`.
2. **Undang anggota dan dosen** sebagai collaborator.
3. **Isi README** dengan nomor dan nama tema, nama dan NIM anggota, serta siapa ketuanya.

Checkpoint minggu 1

No	Yang diselesaikan
1	Proyek KMP kelompok dibuat dari template Compose Multiplatform dan di-push ke repository kelompok
2	Judul aplikasi diganti dengan nama tema, dan nama platform ikut tampil memakai expect dan actual
3	Setiap anggota punya satu commit, misalnya menambahkan nama dan NIM di layar Tentang

Kumpulkan tautan repository kelompok di LMS, dengan README yang berisi tema, anggota, NIM, dan tangkapan layar aplikasi. Checkpoint tidak masuk nilai, tetapi menjadi bekal tes lisan di pertemuan 12 sampai 16. Checkpoint yang tertunda akan menumpuk di sana.

PENUTUP

Rangkuman

Kotlin membedakan nilai yang tetap (`val`) dan nilai yang boleh berubah (`var`). Biasakan memakai `val`. Setiap nilai punya tipe, dan tanda tanya di belakang tipe berarti nilainya boleh kosong.

Fungsi adalah perintah yang diberi nama. Fungsi satu baris ditulis dengan tanda sama dengan. String template memakai `$nama` untuk variabel dan `${...}` untuk ekspresi. Parameter bawaan membuat pemanggilan lebih ringkas.

`when` memilih satu cabang dari atas ke bawah dan bisa menghasilkan nilai. `for` dengan rentang seperti `1..5` mengulang pekerjaan.

Kotlin Multiplatform menulis logika sekali di `commonMain`, lalu menjalankannya di Android, iOS, dan desktop. Kode yang berbeda per platform ditulis di `androidMain`, `iosMain`, dan `desktopMain`, dihubungkan lewat pasangan `expect` dan `actual`.

PENUTUP

Cek pemahaman

Jawab tanpa melihat modul dulu, lalu cocokkan dengan kunci di bawahnya.

1. Kapan kalian memakai `val`, dan kapan `var`? Beri satu contoh untuk masing-masing dari `LaporKampus`.
2. Apa keluaran dari `println("Ada ${2 + 3} laporan")`?
3. Dengan fungsi tingkat di Langkah 3, apa hasil `tingkat(0)`?
4. Apa yang ditulis sekali di `commonMain`, dan apa yang tetap ditulis per platform?
5. Proyek kalian punya `expect fun namaPlatform()` di `commonMain`, tetapi actual-nya hanya ada di `androidMain`. Target mana yang gagal di-build?

JAWABAN Kunci jawaban

1. `val` untuk nilai yang tidak berubah, misalnya nama kampus atau nomor laporan. `var` untuk nilai yang berubah, misalnya jumlah laporan yang terus bertambah.
2. Ada 5 laporan, karena ekspresi di dalam `${ ... }` dihitung lebih dulu.
3. "Biasa". Nilai 0 tidak cocok dengan 5 maupun rentang 3..4, jadi jatuh ke cabang `else`.
4. Logika yang sama untuk semua platform, seperti model data, aturan status, dan sebagian besar UI, ditulis di `commonMain`. Yang memakai fitur bawaan sistem operasi, seperti kamera, izin akses, dan nama perangkat, ditulis per platform.
5. Target iOS dan desktop gagal, karena keduanya belum punya actual. Target Android berhasil.

PENUTUP

Bacaan lanjut dan persiapan minggu depan

Sumber	Untuk apa
kotlinlang.org, Basic syntax	Mengulang val, var, fungsi, dan string template
kotlinlang.org, Conditions and loops	Mendalami if, when, for, dan rentang
kotlinlang.org, Expected and actual declarations	Aturan lengkap expect dan actual
kmp.jetbrains.com	Membuat proyek KMP baru

CONTOH **Masalah Pertemuan 2**

Status laporan berubah dari Baru menjadi Diproses, lalu Selesai. Bagaimana aplikasi tahu saat statusnya berubah, tanpa pengguna menekan tombol muat ulang? Sebelum datang, pastikan aplikasi kelompok bisa di-build di laptop setiap anggota, dan latihan minggu 1 sudah ada di repository latihan pribadi.